# ACTIVIDAD SPRING BOOT

## API REST para la gestión de productos y usuarios de una tienda

Frederick Alexandro Abril Mogollón
Ficha: [FICHA]

Instructor: [INSTRUCTOR]

Programa de formación: [PROGRAMA]

16 de octubre de 2026

## Introducción

En este informe describo el trabajo que hice en el taller de Spring Boot, que consistía en tomar un proyecto base con errores, ponerlo a funcionar y construir sobre él una API REST con el CRUD completo de dos entidades: productos y usuarios. El proyecto se llama demo y conserva el paquete com.example.api.demo con el que venía desde el principio.

El trabajo se dividió en cinco fases. Primero hice un diagnóstico de los errores que impedían compilar y arrancar la aplicación. Después configuré la base de datos, apliqué Lombok para reducir el código repetitivo, organicé el proyecto por capas y por último probé todos los endpoints con Postman siguiendo la matriz de casos de la guía. Al final del documento están las capturas que sirven como evidencia de cada paso.

### Objetivos

El objetivo general fue construir y probar una API REST con Spring Boot que permita crear, consultar, actualizar y eliminar productos y usuarios, guardando la información en una base de datos real.

Como objetivos específicos me propuse encontrar y corregir los errores del proyecto base, conectar la aplicación a una base de datos PostgreSQL que corre en Docker, usar Lombok en las entidades, servicios y controladores, separar el código en capas con un manejo de errores centralizado y comprobar con Postman que cada endpoint responde con el código HTTP correcto.

## Fase 1 – Diagnóstico

Antes de cambiar cualquier cosa hice una copia completa del proyecto original en la carpeta demo_respaldo_original, para poder comparar al final. Luego intenté compilar y ejecutar la aplicación y fui anotando cada error. En la Tabla 1 resumo los tres errores que encontré.

*Tabla 1. Errores encontrados en el diagnóstico*

| N.º | Error | Archivo | Causa | Solución |
|---|---|---|---|---|
| 1 | FATAL: la autentificación password falló para el usuario «tiendaya» y luego Unable to determine Dialect | application.properties y docker-compose.yml | El puerto 5432 ya lo usa el PostgreSQL instalado en Windows, así que la aplicación se conectaba a ese servidor y no al contenedor | Publiqué el contenedor en el puerto 5433 y cambié la URL de conexión; los datos se pueden cambiar con variables de entorno |
| 2 | mvn clean install fallaba en la prueba contextLoads | DemoApplicationTests.java | La prueba necesitaba una base de datos real para levantar el contexto de Spring | Agregué H2 en memoria solo para las pruebas, con su propio application.properties |
| 3 | Las rutas no coincidían con las que pide la guía | ProductController.java | Usaba /api/products y recibía los datos con @RequestParam en lugar de un cuerpo JSON | Lo reescribí como ProductoController en /api/productos y con @RequestBody |

El primer error fue el que más tiempo me tomó entender. El mensaje decía que la contraseña del usuario tiendaya era incorrecta, pero la contraseña estaba bien escrita. Revisando los puertos ocupados me di cuenta de que en mi computador ya había un PostgreSQL instalado escuchando en el puerto 5432, de modo que la aplicación nunca llegaba al contenedor de Docker. La solución fue publicar el contenedor en el puerto 5433 y apuntar la URL de conexión a ese puerto.

El segundo error apareció al ejecutar mvn clean install. La única prueba del proyecto intentaba levantar todo el contexto de Spring, y eso incluye conectarse a la base de datos. Para que las pruebas no dependan de que Docker esté encendido agregué la dependencia de H2 con alcance de pruebas y un archivo application.properties dentro de src/test/resources que usa una base en memoria.

El tercer problema no impedía compilar, pero hacía que la API no cumpliera lo que pide la guía. El controlador original estaba en inglés, usaba la ruta /api/products y esperaba el nombre y el precio como parámetros de la URL. Lo reemplacé por un controlador que trabaja en /api/productos y recibe un JSON en el cuerpo de la petición, que es como se usa normalmente una API REST.

## Fase 2 – Base de datos

Para la base de datos usé el archivo docker-compose.yml que ya traía el proyecto. Este archivo crea un contenedor llamado tiendaya-db con PostgreSQL 17,

una base llamada tiendaya y un volumen para que los datos no se pierdan al apagar el contenedor. En application.properties dejé la URL, el usuario y la contraseña como variables de entorno con valores por defecto de desarrollo, así no queda ninguna contraseña real escrita en el código. Las tablas productos y usuarios las crea Hibernate automáticamente gracias a la propiedad ddl-auto=update.

**Nota:** la guía menciona MySQL, pero yo usé PostgreSQL porque era la base de datos que ya tenía configurada el proyecto base, junto con su contenedor de Docker. El funcionamiento de JPA es el mismo con cualquiera de las dos y solo cambian el driver y la URL de conexión.

## Fase 3 – Lombok

En esta fase apliqué Lombok para quitar el código repetitivo. En las entidades Producto y Usuario usé @Getter, @Setter, @NoArgsConstructor, @AllArgsConstructor y @Builder. En los servicios y controladores reemplacé los constructores escritos a mano por @RequiredArgsConstructor, que genera el constructor con los atributos final y permite que Spring inyecte las dependencias sin usar @Autowired. En los servicios también agregué @Slf4j para tener un logger.

Para ver la diferencia comparo el servicio de productos del respaldo original con el servicio actual. Antes el constructor había que escribirlo a mano:

```
// Antes: ProductService.java (respaldo original)
@Service
public class ProductService {

    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }
```

Después, con Lombok, la clase solo declara el atributo y la anotación se encarga del resto:

```
// Después: ProductoService.java (proyecto actual)
@Service
@RequiredArgsConstructor
@Slf4j
public class ProductoService {

    private final ProductoRepository productoRepository;
```

Lo mismo pasó con las entidades. La clase Product original solo tenía @Getter, así que para actualizar un producto había que crear un objeto nuevo con todos sus datos. Ahora Producto tiene también @Setter y @Builder, lo que permite modificar solo los campos que cambian.

## Fase 4 – Módulos de productos y usuarios

Rehice el módulo de productos agregando los campos descripción y stock, y construí desde cero el módulo de usuarios con nombre, correo, contraseña y rol. Cada módulo tiene sus cinco endpoints: listar, consultar por id, crear, actualizar y eliminar. Los datos se validan con anotaciones como @NotBlank, @Email y @PositiveOrZero, y si llega algo inválido la API responde 400 con un mensaje por cada campo.

El módulo de usuarios tiene dos reglas de negocio. La primera es que el correo no se puede repetir: si intento crear o actualizar un usuario con un correo que ya existe, la API responde 400 con el mensaje «Ya existe un usuario con el correo ...». La segunda es que la contraseña se recibe en las peticiones pero nunca se devuelve en las respuestas. Para eso usé @JsonProperty(access = WRITE_ONLY) sobre el campo contrasena.

## Fase 5 – Pruebas

Probé la API de dos formas. Primero escribí la clase ApiCrudTests, que recorre con MockMvc los casos de la matriz para los dos módulos y se ejecuta sola cada vez que hago mvn clean install. Después importé la colección postman/API_Tienda.postman_collection.json en mi workspace de Postman, creé un environment con mi nombre y ejecuté cada petición a mano con la API conectada a PostgreSQL. Cada petición de la colección tiene un test que verifica el código HTTP esperado.

## Estructura por capas del proyecto

El código quedó organizado en cinco paquetes dentro de com.example.api.demo, y cada uno tiene una responsabilidad distinta. En la Tabla 2 explico qué contiene cada capa.

*Tabla 2. Capas del proyecto*

| Capa (paquete) | Clases | Responsabilidad |
|---|---|---|
| controller | ProductoController, UsuarioController, HomeController | Recibe las peticiones HTTP, valida el cuerpo con @Valid y devuelve la respuesta con el código adecuado (200, 201, 204) |
| service | ProductoService, UsuarioService | Contiene la lógica de negocio, como verificar que el registro exista o que el correo no esté repetido |
| repository | ProductoRepository, UsuarioRepository | Interfaces que extienden JpaRepository y hacen las consultas a la base de datos |
| model | Producto, Usuario | Entidades JPA con sus validaciones; cada una corresponde a una tabla |
| exception | RecursoNoEncontradoException, ManejadorGlobalExcepciones | Convierte los errores en respuestas JSON con código 404 o 400 |

El flujo de una petición siempre es el mismo: el controlador recibe la petición y llama al servicio, el servicio aplica las reglas y usa el repositorio para leer o guardar en la base de datos, y si algo falla lanza una excepción que el manejador global transforma en una respuesta con el mensaje de error.

## Matriz de pruebas

La Tabla 3 muestra los catorce casos de la guía con el código que esperaba y el que obtuve. Los casos 1 a 10 los ejecuté para los dos módulos. El caso 11 lo comprobé con la prueba automática y con una petición directa, y el caso 12 deteniendo y volviendo a arrancar la API.

*Tabla 3. Matriz de pruebas*

| Caso | Descripción | Método | Endpoint | Esperado | Obtenido |
|---|---|---|---|---|---|
| 1 | Crear registro válido | POST | /api/productos, /api/usuarios | 201 | 201 |
| 2 | Crear segundo registro | POST | /api/productos, /api/usuarios | 201 | 201 |
| 3 | Listar todos | GET | /api/productos, /api/usuarios | 200 | 200 |
| 4 | Consultar id existente | GET | /api/{recurso}/1 | 200 | 200 |
| 5 | Consultar id inexistente | GET | /api/{recurso}/999 | 404 | 404 |
| 6 | Actualizar existente | PUT | /api/{recurso}/1 | 200 | 200 |
| 7 | Actualizar inexistente | PUT | /api/{recurso}/999 | 404 | 404 |
| 8 | Eliminar existente | DELETE | /api/{recurso}/2 | 204 | 204 |
| 9 | Eliminar inexistente | DELETE | /api/{recurso}/999 | 404 | 404 |
| 10 | Listar después de eliminar | GET | /api/productos, /api/usuarios | 200 | 200 |
| 11 | Crear con datos inválidos | POST | /api/productos | 400 | 400 |
| 12 | Persistencia tras reiniciar la API | GET | /api/productos, /api/usuarios | 200 | 200 |
| 13 | Crear usuario con correo duplicado | POST | /api/usuarios | 400 | 400 |
| 14 | La contraseña no aparece en la respuesta | GET | /api/usuarios/1 | 200 | 200 |

## Evidencias

A continuación están las capturas en el orden en que hice las pruebas. En las de Postman se ve mi workspace y el environment «Frederick Abril Mogollón», que define la variable estudiante con mi nombre y la variable baseUrl con el valor http://localhost:8080. En las de la terminal mi nombre aparece en el título de la ventana.

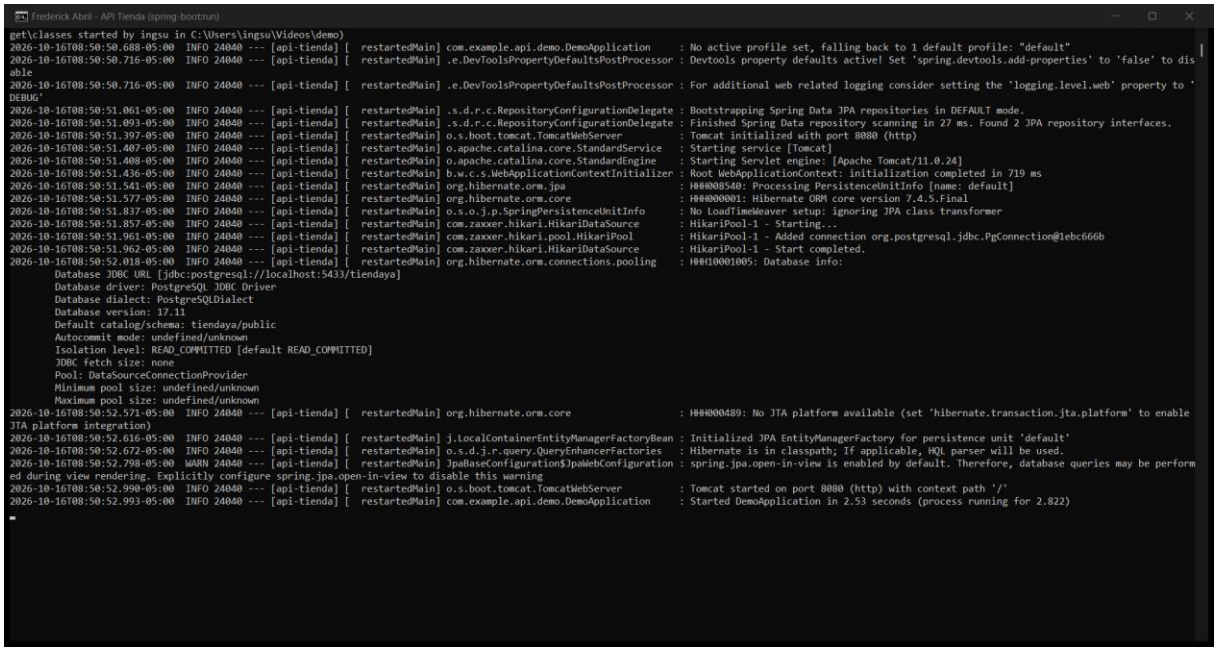



**Figura 1.** *Arranque de la API con mvnw spring-boot:run; se ve la conexión a PostgreSQL en el puerto 5433 y el mensaje Started DemoApplication con fecha y hora.*

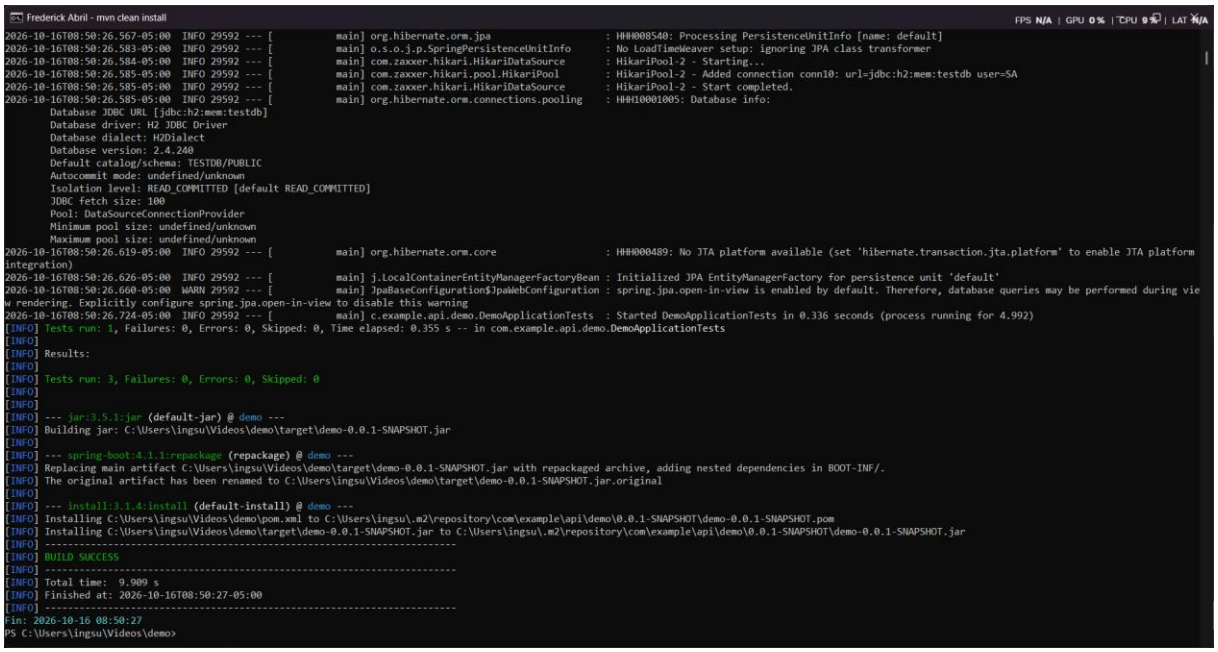



**Figura 2.** *Resultado de mvn clean install: 3 pruebas sin fallos y BUILD SUCCESS.*

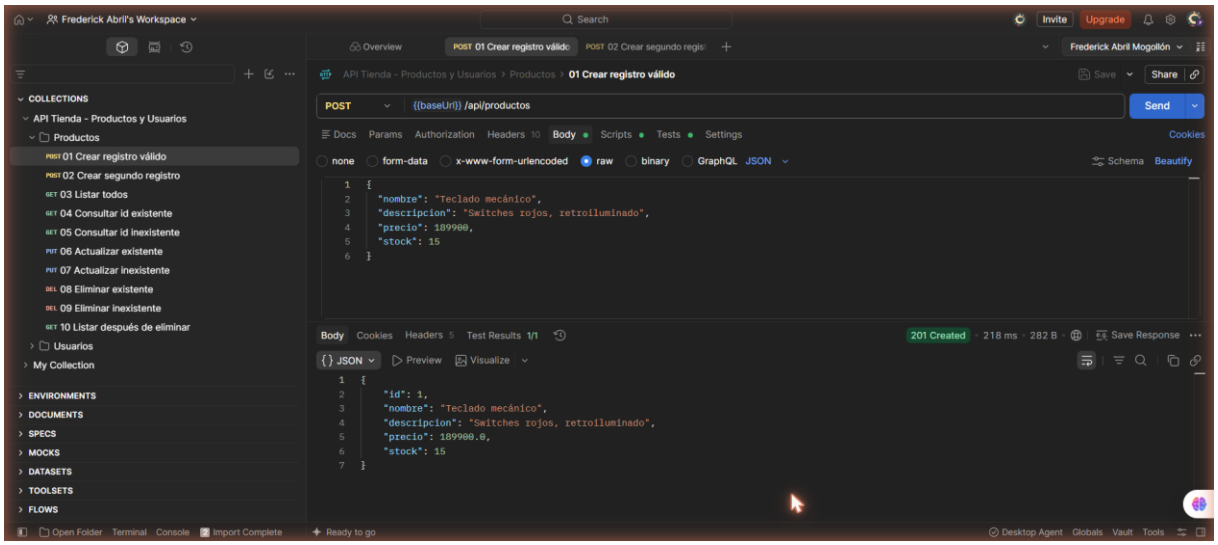



**Figura 3.** *Caso 1 de productos: POST /api/productos crea el producto con id 1 y responde 201 Created.*

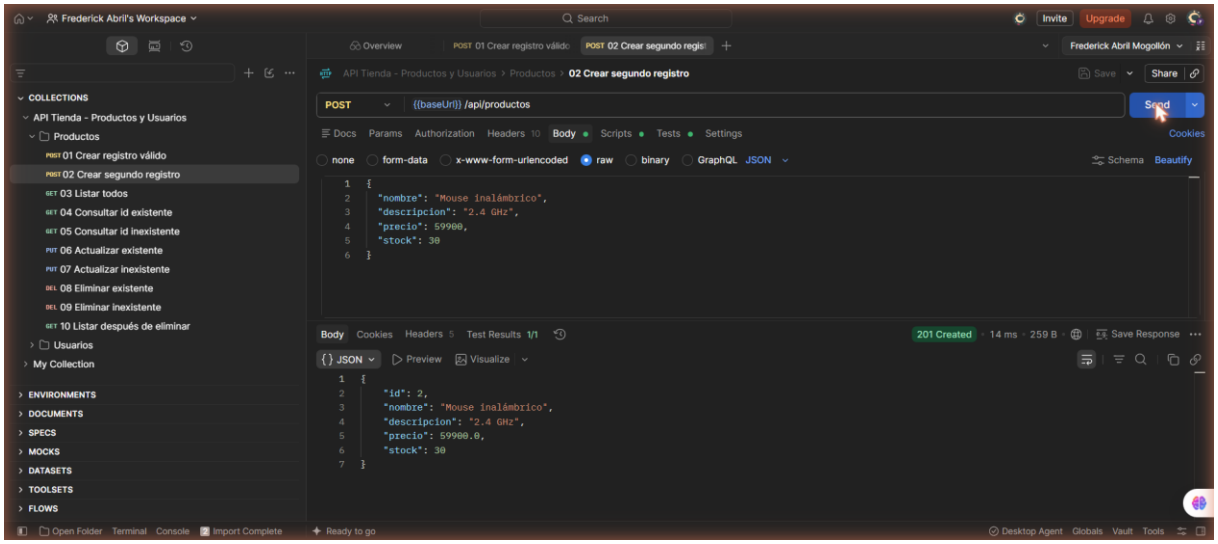



**Figura 4.** *Caso 2 de productos: se crea un segundo producto con id 2 (201).*

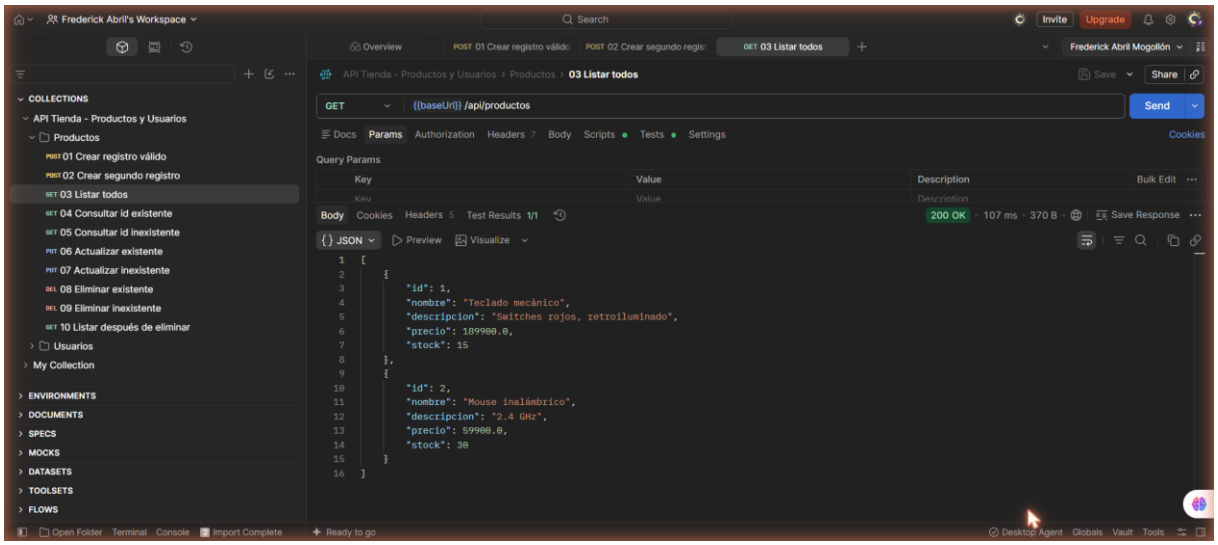



**Figura 5.** *Caso 3 de productos: GET /api/productos devuelve los dos productos (200).*

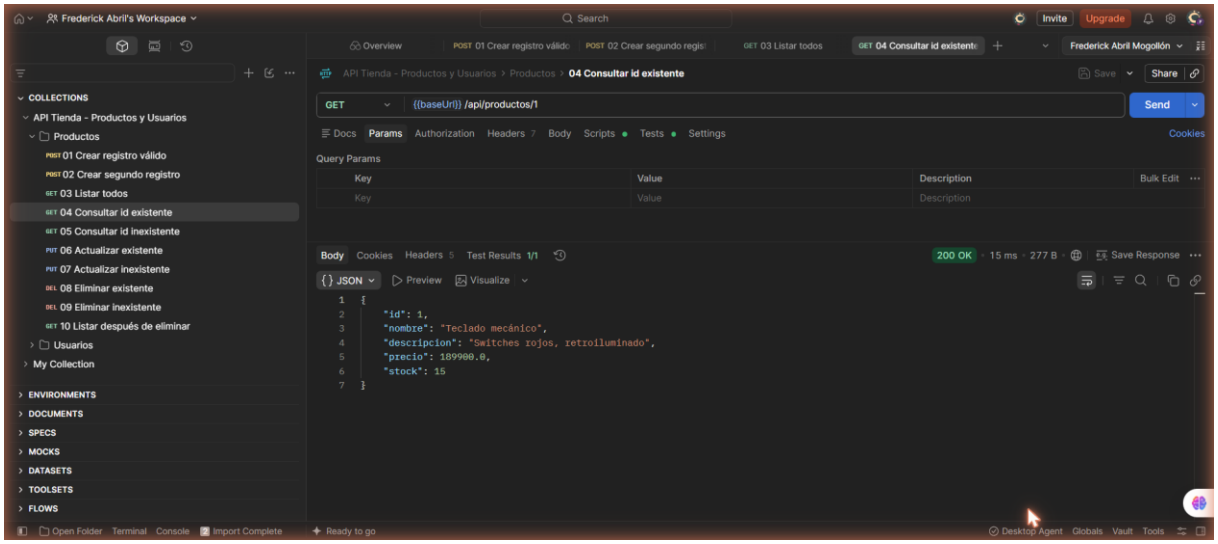



**Figura 6.** *Caso 4 de productos: GET /api/productos/1 devuelve el producto (200).*

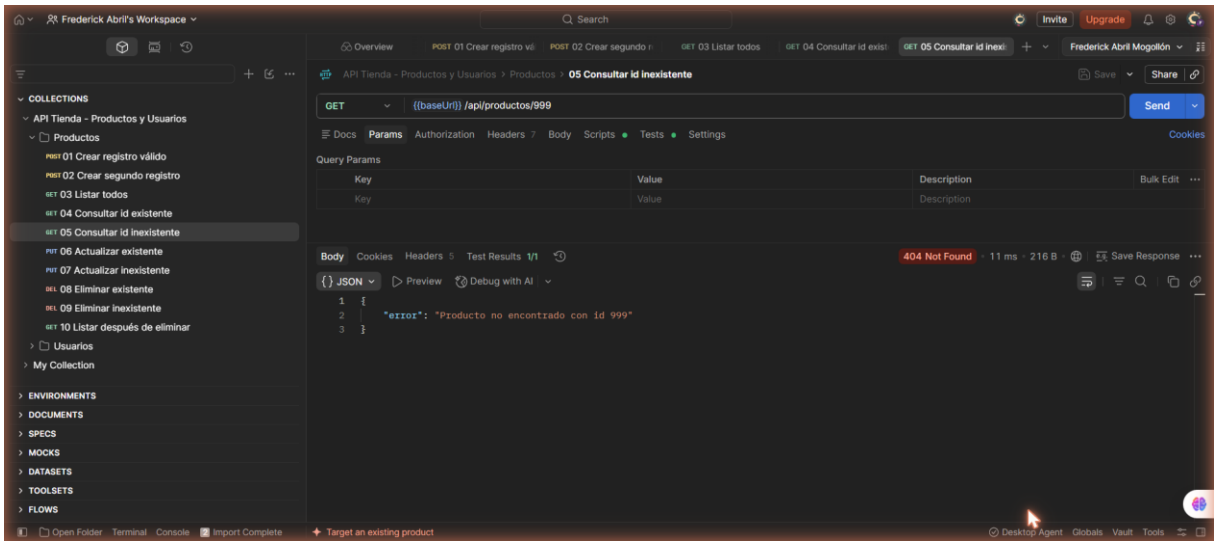



**Figura 7.** *Caso 5 de productos: GET /api/productos/999 responde 404 con el mensaje de error.*

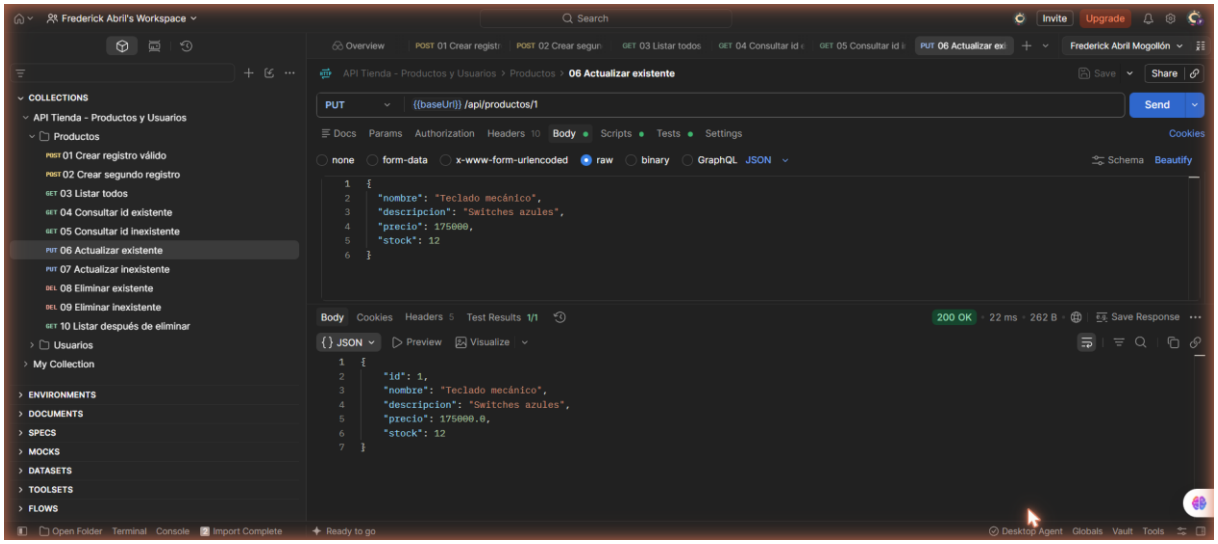



**Figura 8.** *Caso 6 de productos: PUT /api/productos/1 cambia la descripción, el precio y el stock (200).*

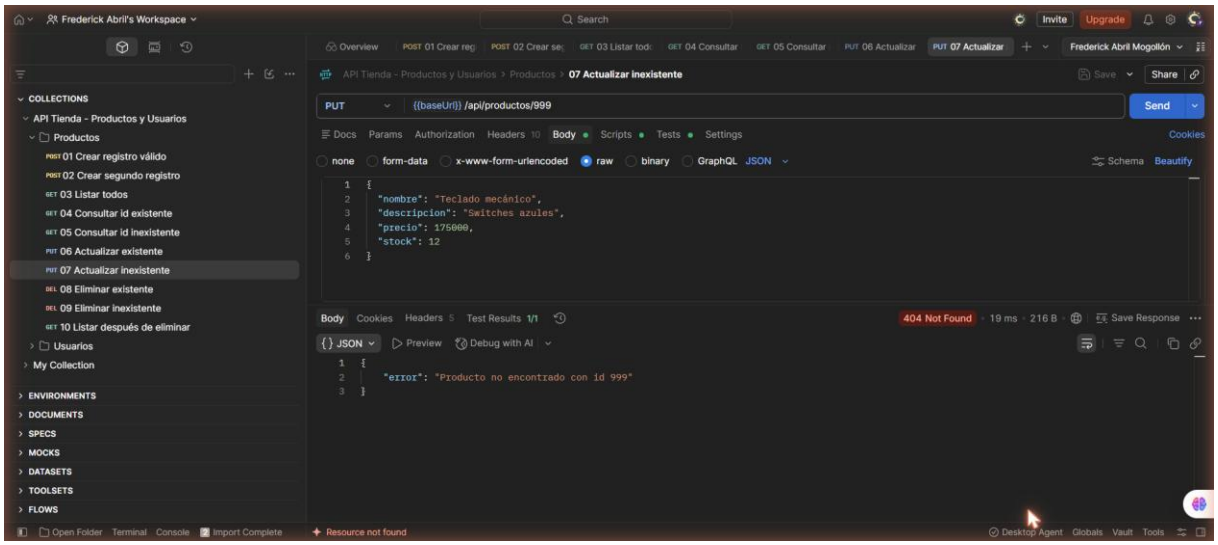



**Figura 9.** *Caso 7 de productos: PUT /api/productos/999 responde 404.*

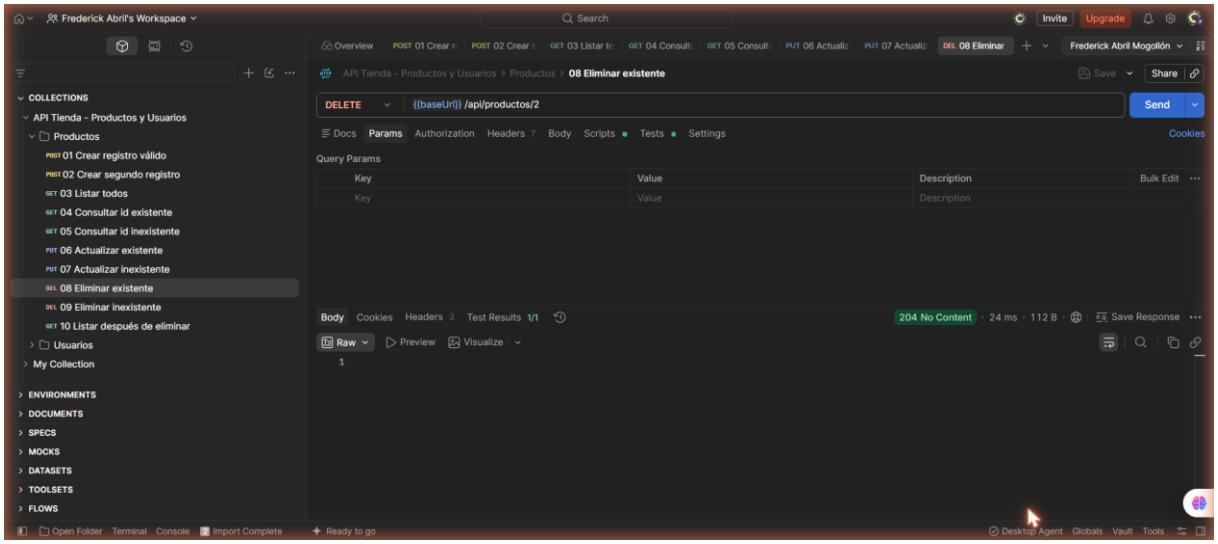



**Figura 10.** *Caso 8 de productos: DELETE /api/productos/2 responde 204 No Content.*

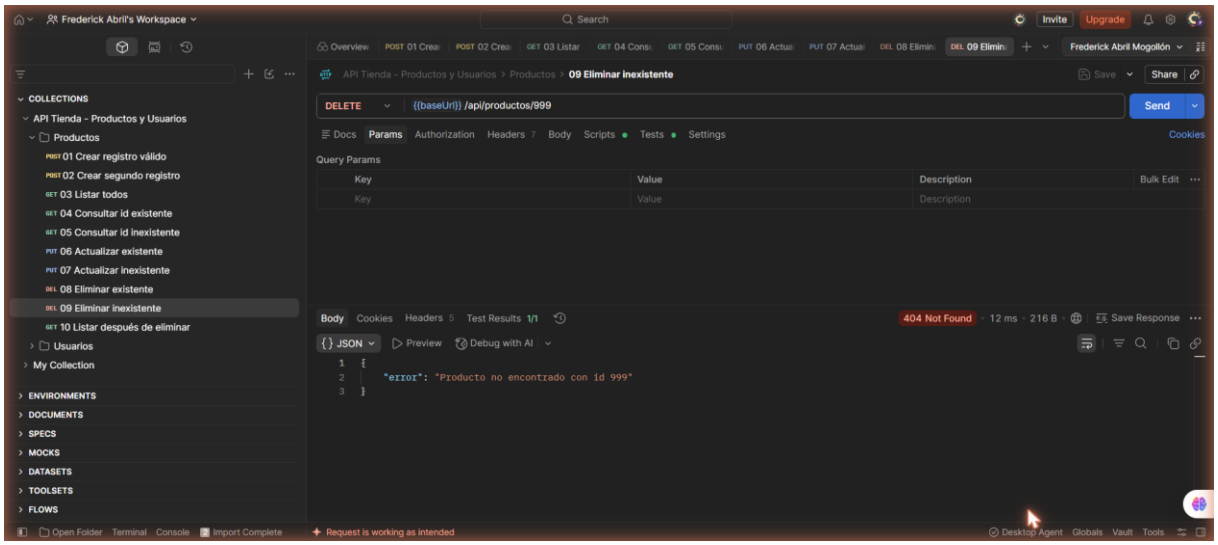



**Figura 11.** *Caso 9 de productos: DELETE /api/productos/999 responde 404.*

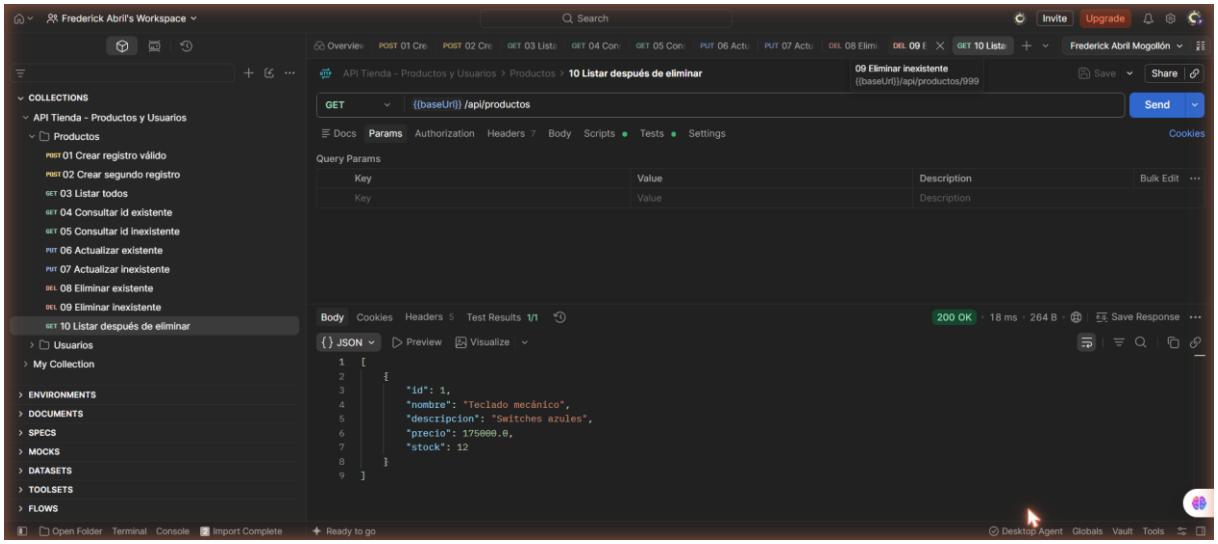



**Figura 12.** *Caso 10 de productos: después de eliminar solo queda el producto con id 1 (200).*

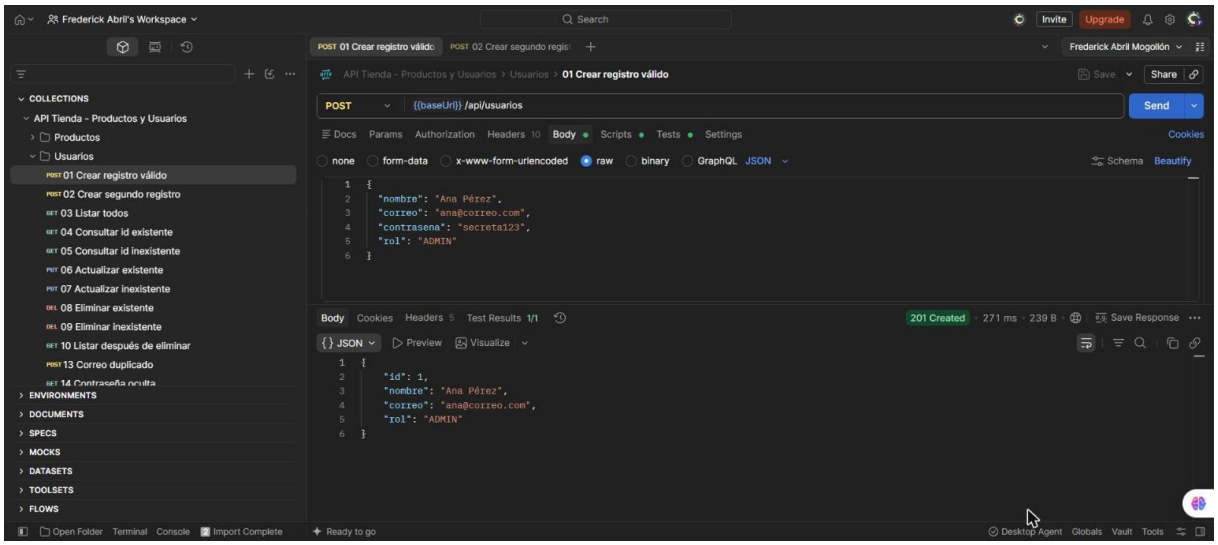



**Figura 13.** *Caso 1 de usuarios: POST /api/usuarios crea a Ana Pérez (201); la respuesta no incluye la contraseña.*

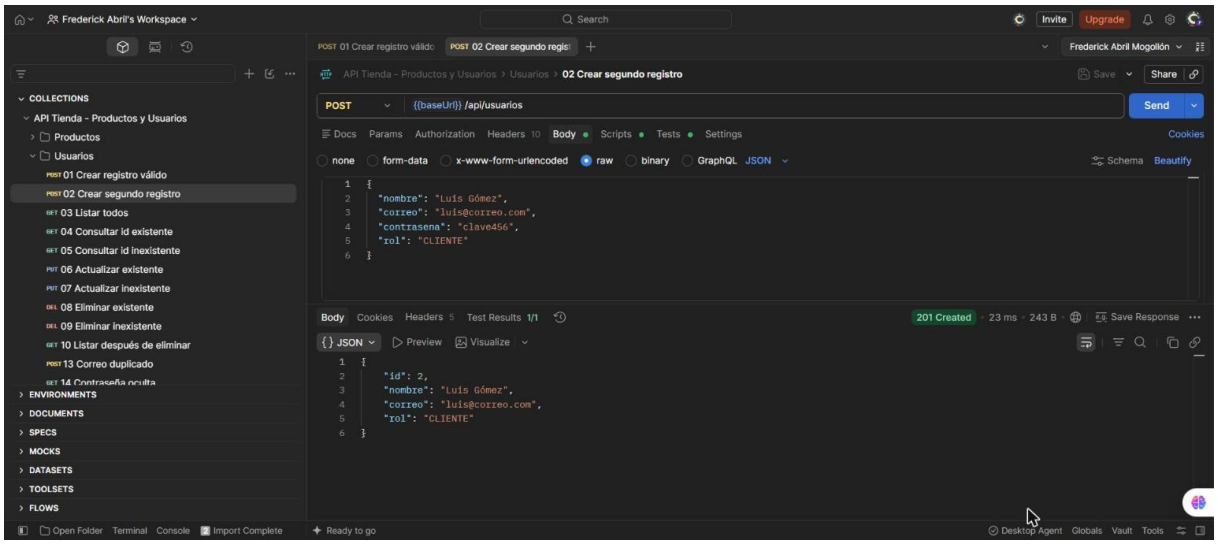



**Figura 14.** *Caso 2 de usuarios: se crea a Luis Gómez con id 2 (201).*

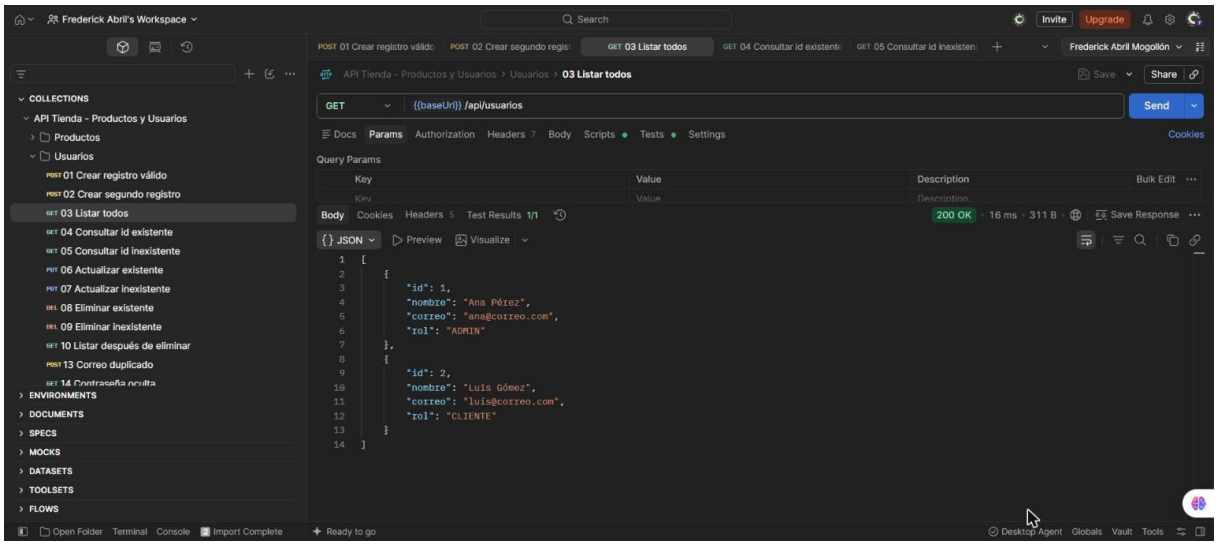



**Figura 15.** *Caso 3 de usuarios: GET /api/usuarios devuelve los dos usuarios sin contraseña (200).*

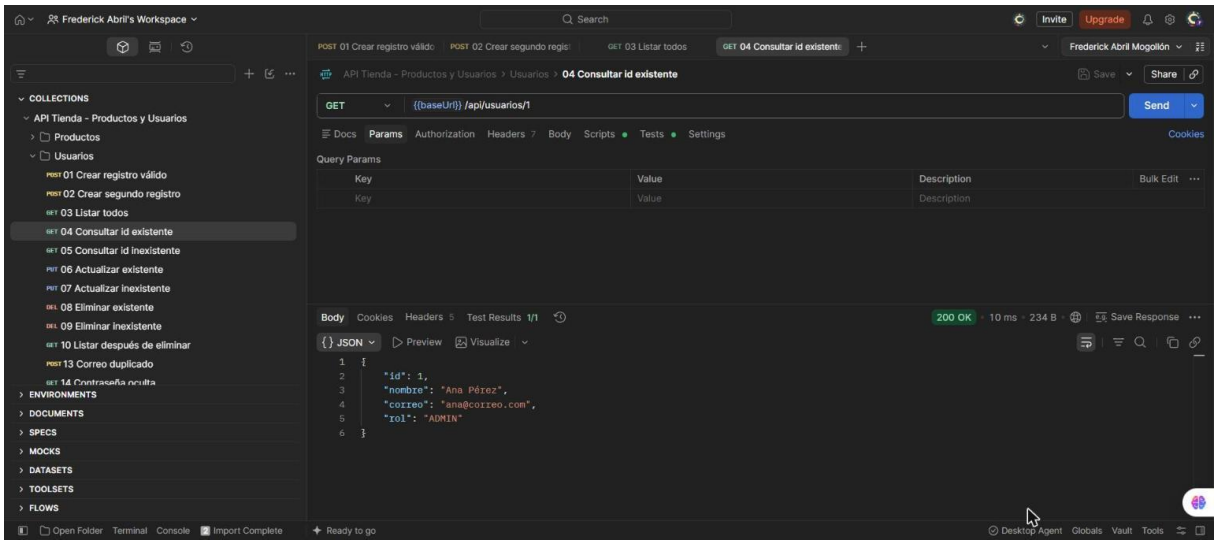



**Figura 16.** *Caso 4 de usuarios: GET /api/usuarios/1 (200).*

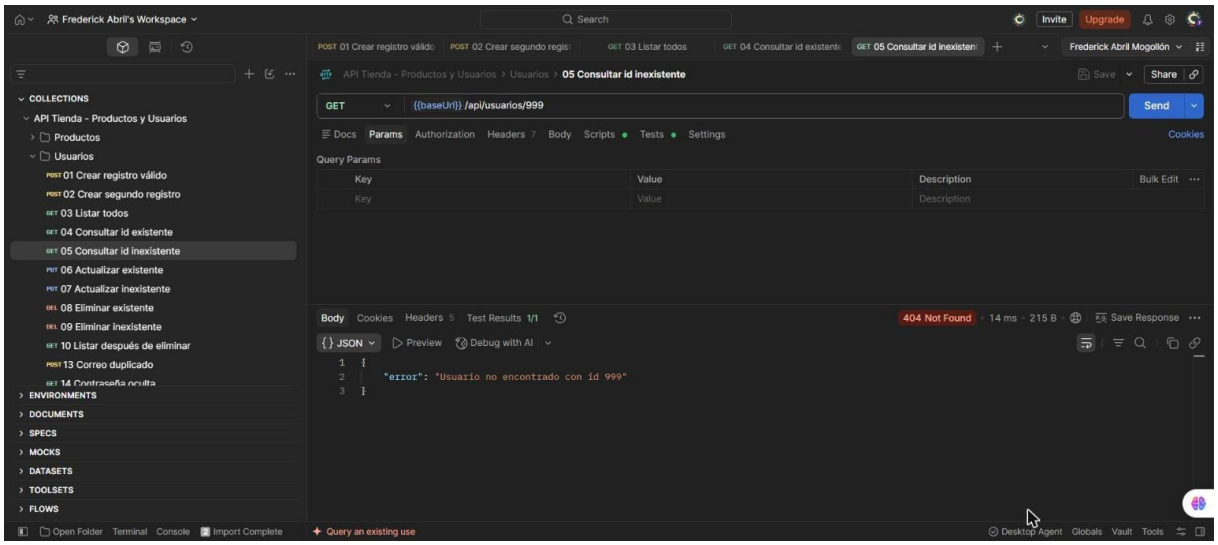



**Figura 17.** *Caso 5 de usuarios: GET /api/usuarios/999 responde 404.*

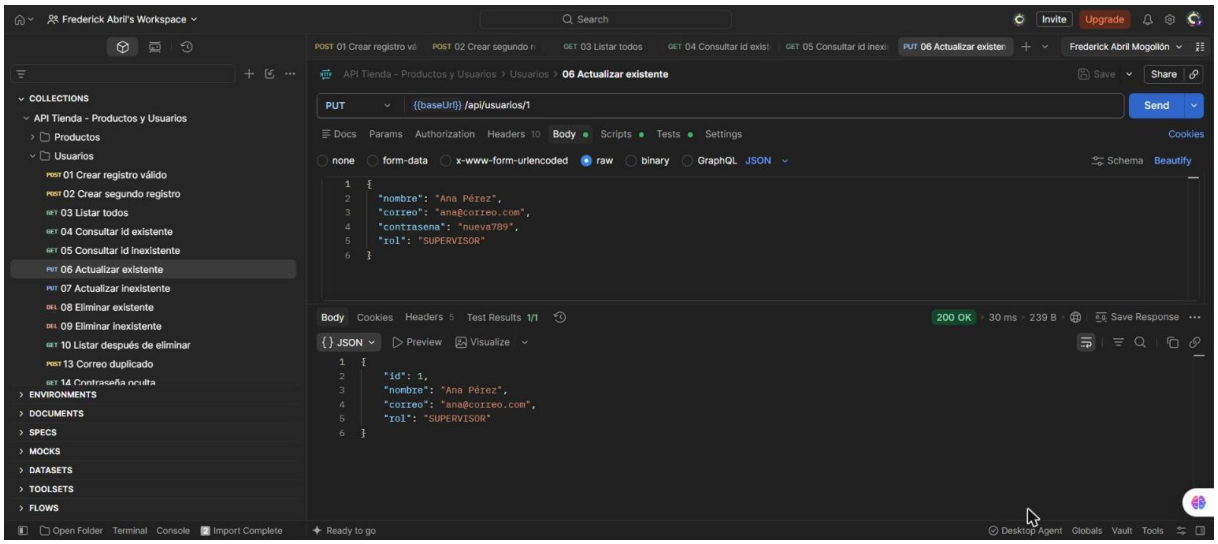



**Figura 18.** *Caso 6 de usuarios: PUT /api/usuarios/1 cambia el rol a SUPERVISOR (200).*

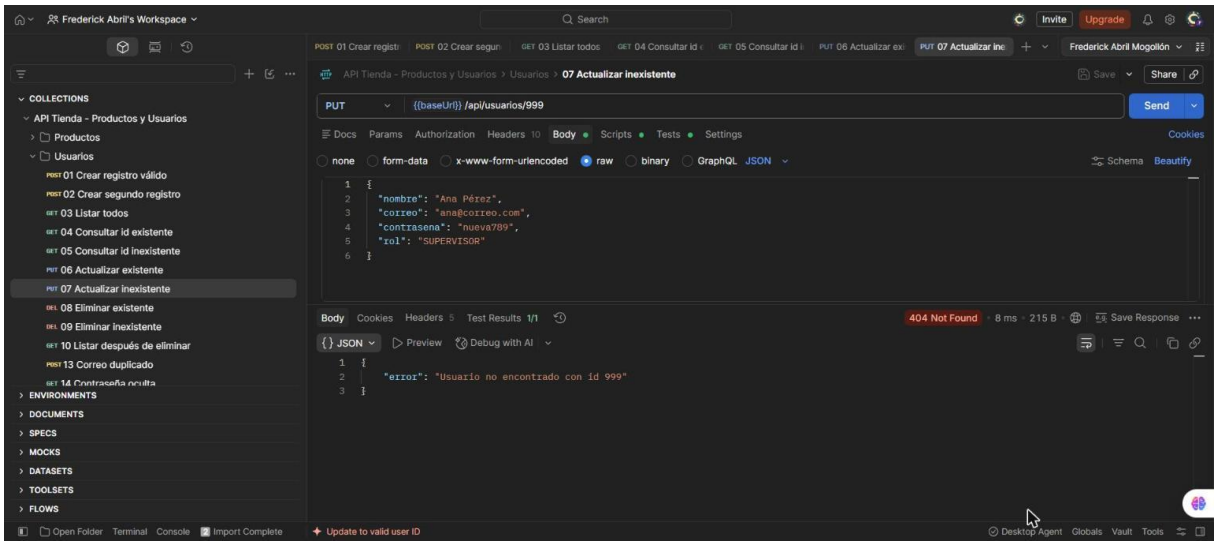



**Figura 19.** *Caso 7 de usuarios: PUT /api/usuarios/999 responde 404.*

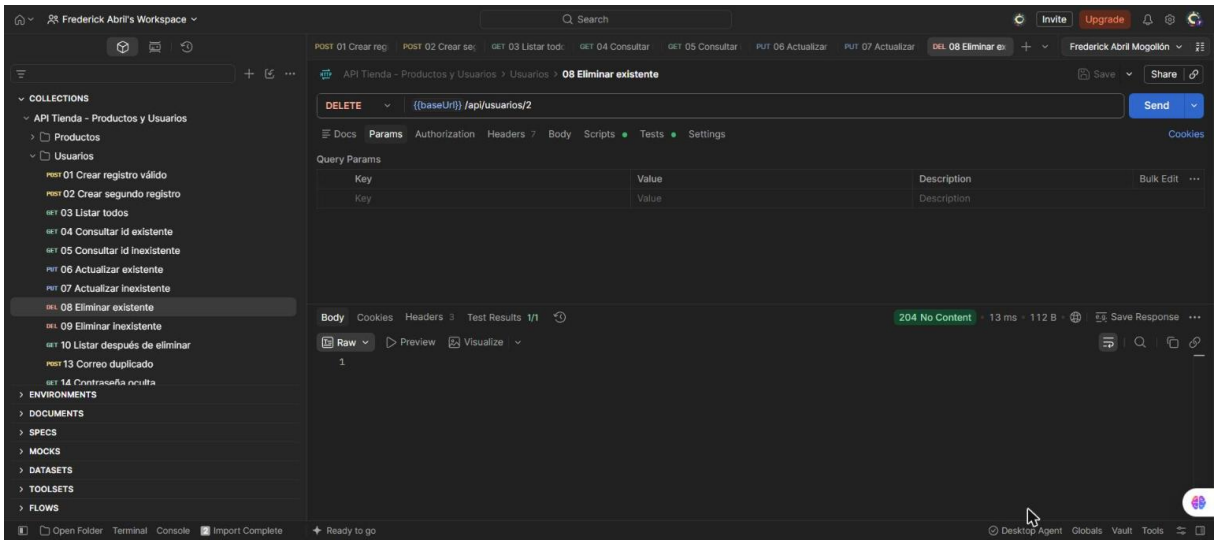



**Figura 20.** *Caso 8 de usuarios: DELETE /api/usuarios/2 responde 204.*

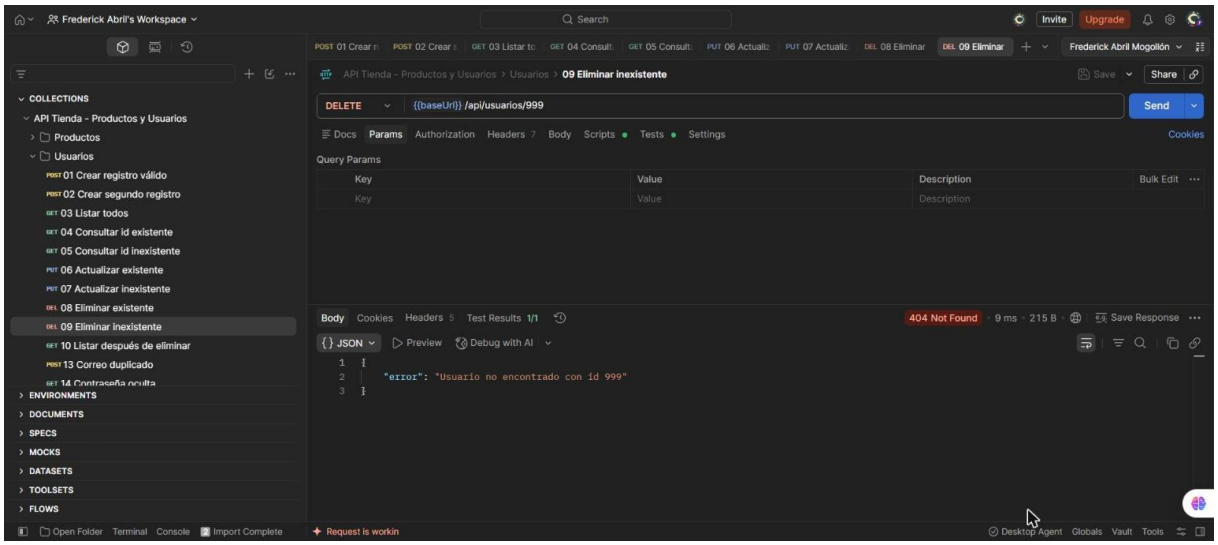



**Figura 21.** *Caso 9 de usuarios: DELETE /api/usuarios/999 responde 404.*

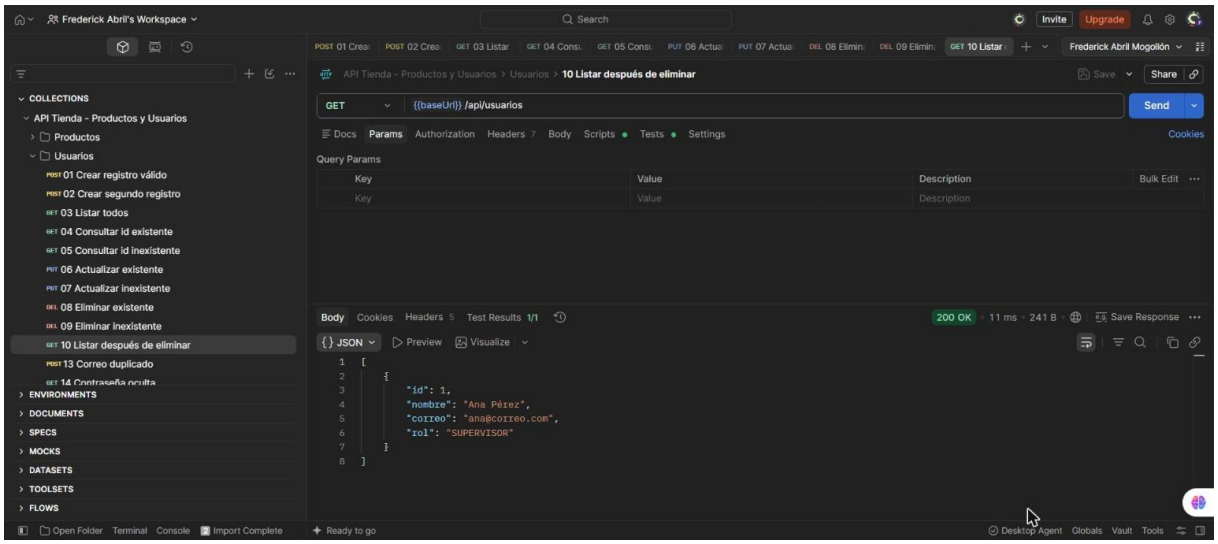



**Figura 22.** *Caso 10 de usuarios: después de eliminar solo queda Ana Pérez (200).*

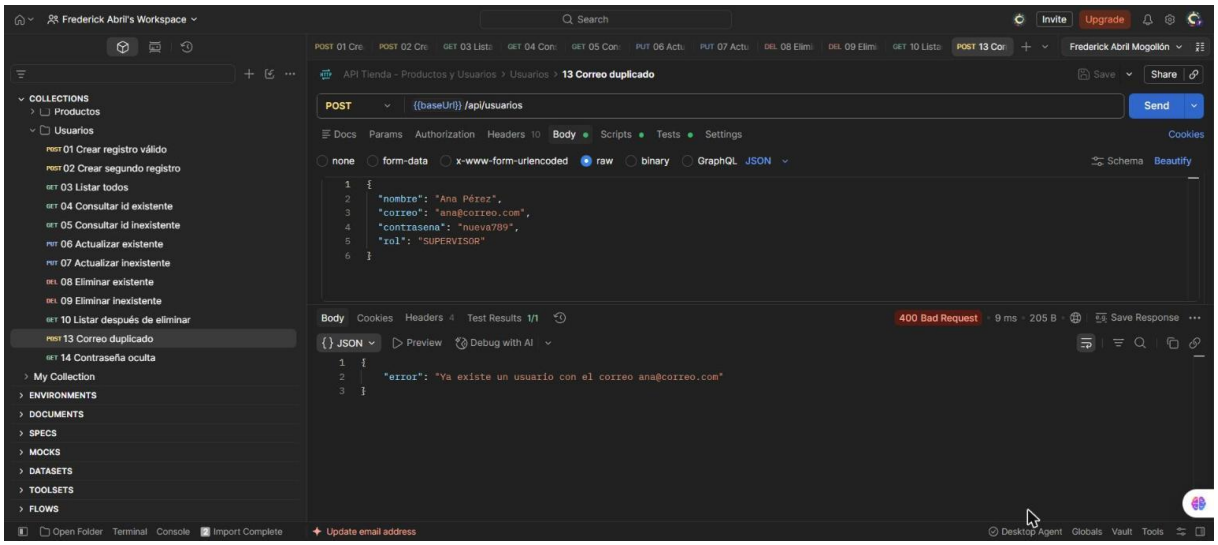



**Figura 23.** *Caso 13: crear un usuario con un correo ya registrado responde 400 Bad Request.*

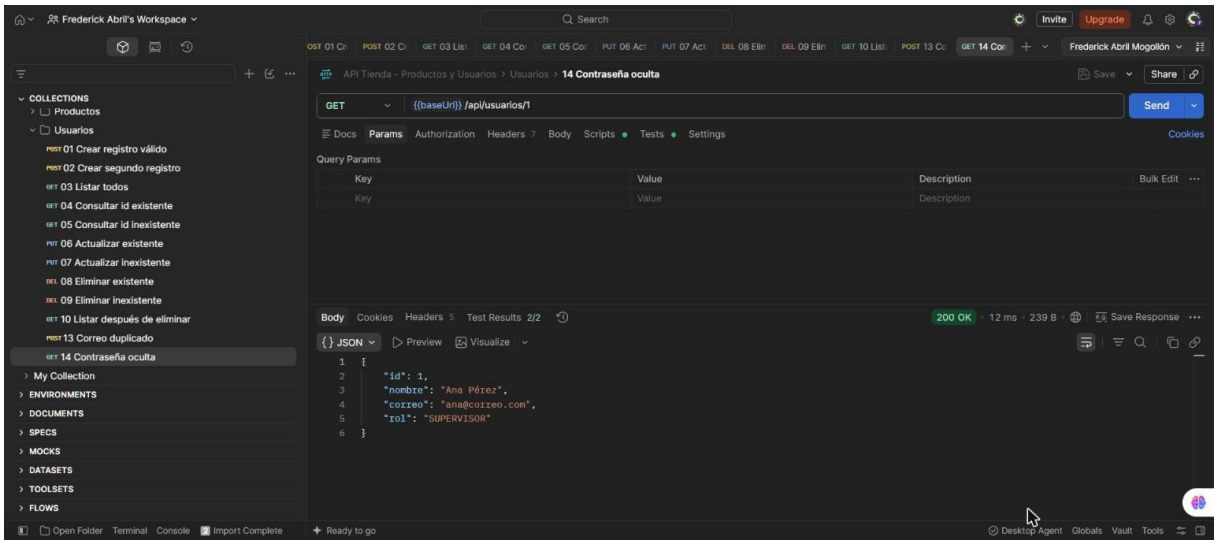



**Figura 24.** *Caso 14: GET /api/usuarios/1 responde 200 y el JSON no contiene el campo contrasena.*

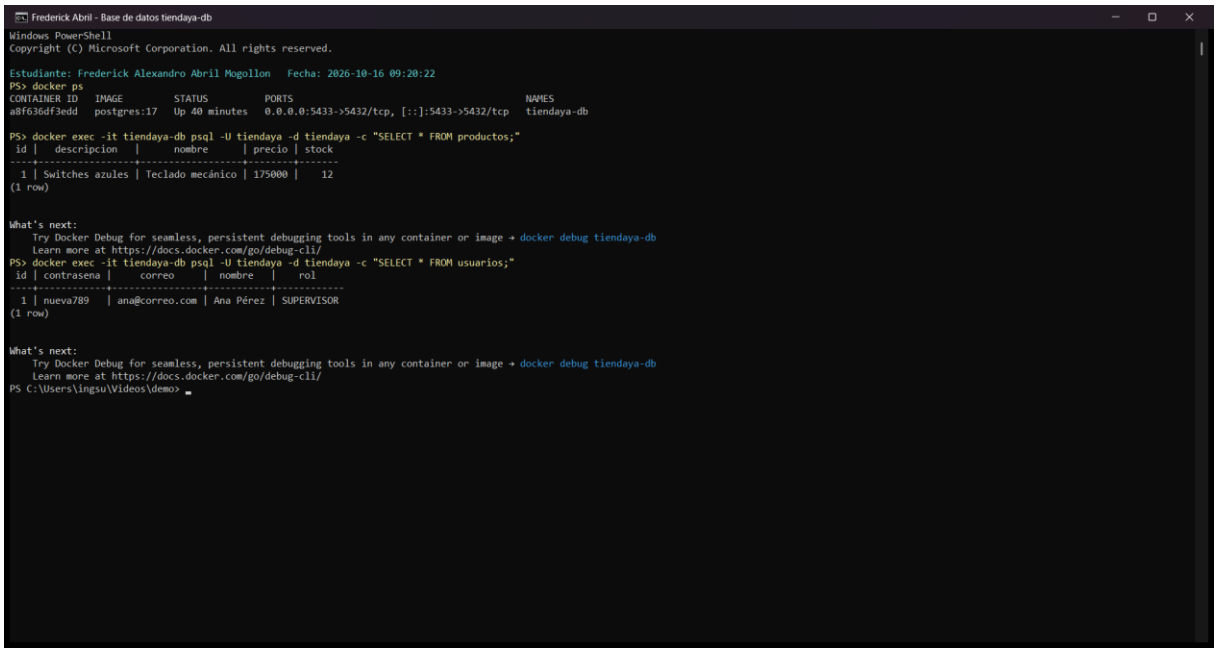



**Figura 25.** *docker ps muestra el contenedor tiendaya-db y las consultas SELECT sobre las tablas productos y usuarios dentro del contenedor.*

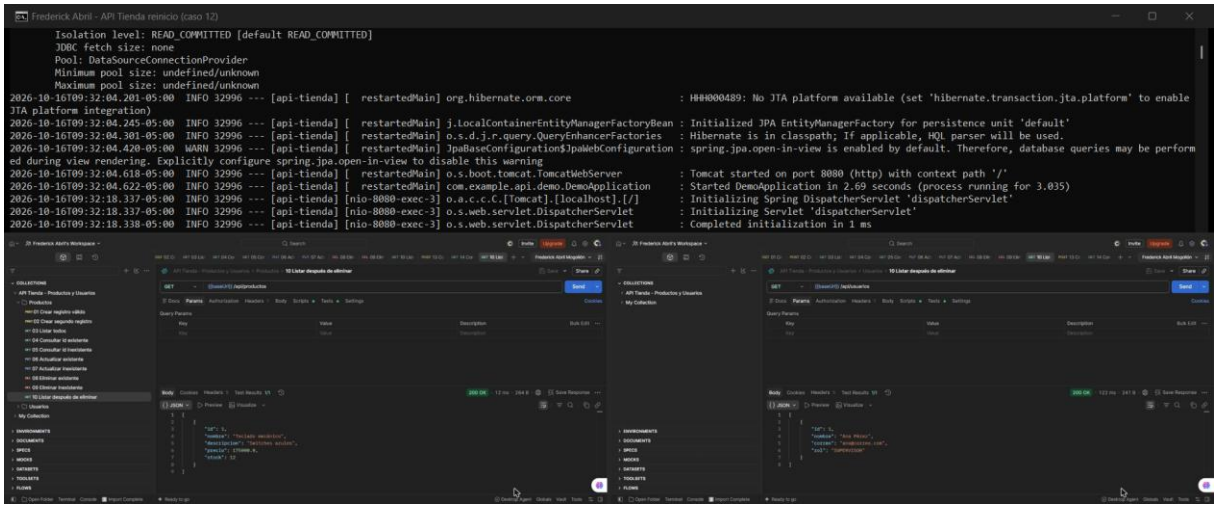



**Figura 26.** *Caso 12: después de detener y volver a arrancar la API, las consultas en Postman devuelven los mismos datos (200).*

En la Figura 25 se ve que en la tabla usuarios la contraseña está guardada como texto plano. La API no la muestra en las respuestas, pero en un sistema real habría que guardarla cifrada, por ejemplo con BCrypt. Lo dejo anotado como una mejora pendiente.

## Conclusiones

Con este taller entendí que muchos errores no están en el código sino en el entorno. El problema del puerto 5432 me enseñó a revisar qué servicios están corriendo en el equipo antes de culpar a la configuración de la aplicación, y a dejar los datos de conexión en variables de entorno para poder cambiarlos sin tocar el código.

Lombok redujo bastante el tamaño de las clases y las hizo más fáciles de leer, sobre todo en los servicios y controladores, donde el constructor ahora lo genera la anotación. También vi la utilidad de separar el proyecto por capas, porque cuando necesité agregar la regla del correo duplicado solo tuve que tocar el servicio de usuarios.

Por último, probar cada caso en Postman y además tener las pruebas automáticas me dio confianza en que la API responde con los códigos correctos. La prueba de persistencia confirmó que los datos quedan guardados en PostgreSQL aunque la aplicación se detenga.

## Repositorio

El código del proyecto, la colección de Postman y las evidencias están publicados en GitHub: https://github.com/Fredpuff/api-tienda-springboot